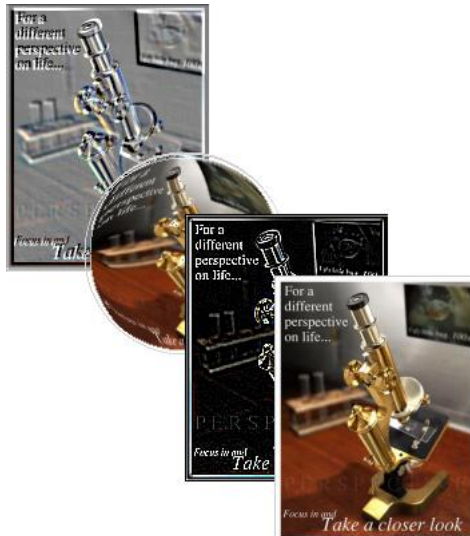


Compressão de Imagens e Formatos de Arquivos



Carlos Alexandre Barros de Mello
CIn/UFPE



Introdução

- Compressão de Imagens
 - Espaço de Armazenamento
 - Velocidade de Transmissão
- Teoria de Códigos
 - Teoria da Informação
 - Claude Shannon [1930]
- Algoritmos de Compressão
 - Estáticos
 - Dinâmicos

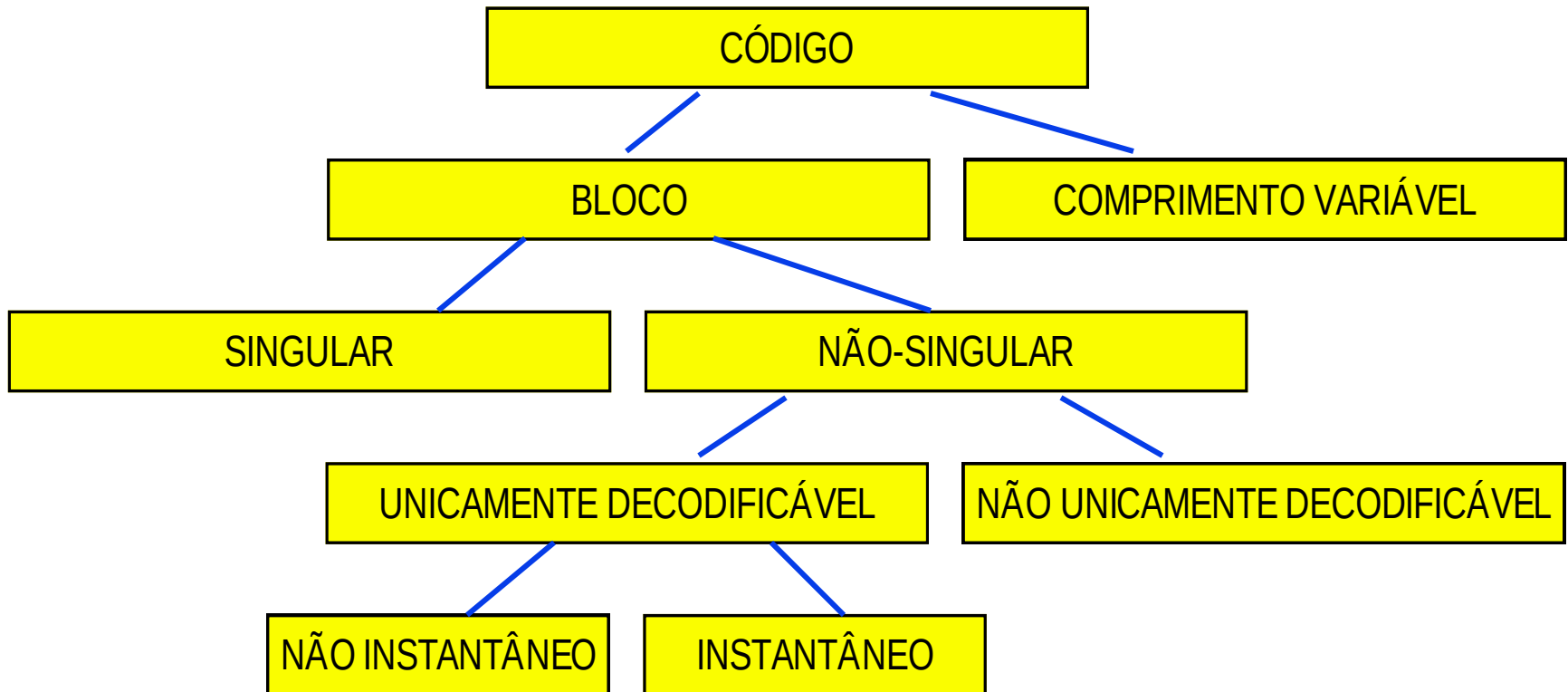
Algoritmos de Compressão

- Sem Perdas
 - Run-Length
 - Huffman
 - Lempel-Ziv
- Com Perdas
 - Wavelets
 - JPEG

Teoria de Códigos

- De Comprimento Variável
- De bloco
 - Singular (palavras-código são distintas)
 - Não-singular
 - Não-unicamente decodificáveis
 - Unicamente decodificáveis
 - Instantâneos
 - Não-instantâneos

Teoria de Códigos



Teoria de Códigos

- Bloco

- $A = 00$
- $B = 010$

- Comprimento variável

- $A = 00$ ou 1
- O comprimento pode variar dentro na codificação de uma mesma palavra

Teoria de Códigos

- Não singulares

- Todas as palavras do código são distintas
- $A = 00$
- $B = 01$

- Singulares

- $A = 00$
- $B = 00$
- $C = 01$

Duas (ou mais) palavras são codificadas com o mesmo código.

Teoria de Códigos

- Não-Unicamente Decodificável
 - $A = 0$
 - $B = 1$
 - $C = 01$
 - Como decodificar a sequência: 0101
 - $0 = A, 1 = B, 01 = C$ ou
 - $01 = C, 01 = C, \text{ etc}$
- Unicamente Decodificável

Teoria de Códigos

- Instantâneo

- Não depende de informações futuras

- Não-Instantâneo

- Depende de informações futuras

$A = 10$ $B = 100$ $C = 1000$ $D = 10000$

Como decodificar 101001000?

Ao receber '10', precisa esperar o próximo símbolo: se for 1, decodifica o 10. Se for 0, ainda pode ser 100, 1000 ou 10000.

Prefixo

- Uma condição necessária e suficiente para um código ser unicamente decodificável é que nenhuma palavra completa do código seja **prefixo** de outra palavra qualquer.

Desigualdade de Kraft

- Garante que um código com palavras com comprimento L_i é **instantâneo**

$$\sum_{i=1}^q r^{-L_i} \leq 1$$

Exemplo:

Considere o código

A = 0 → comprimento $L_i=1$

B = 01 → comprimento $L_i=2$

C = 11 → comprimento $L_i=2$

Número binário → base $r=2$

Ou seja,

$$2^{-1} + 2^{-2} + 2^{-2} = 1$$

Desigualdade de McMillan

- Garante que um código com palavras com comprimento L_i é unicamente decodificável

$$\sum_{i=1}^q r^{-L_i} \leq 1$$

Comprimento Médio

$$L = \sum_{i=1}^q P_i * L_i$$

- Um código é dito **compacto** para uma fonte S se seu comprimento médio é menor ou igual ao comprimento médio de todos os outros códigos unicamente decodificáveis para a mesma fonte S.

Taxa de Compressão

- Mas, usualmente, medimos a compressão tomando como base a relação entre o tamanho do arquivo original e o do compactado (tamanhos em Kbytes, por exemplo)

$$C = \frac{\text{Tamanho do Arquivo Original}}{\text{Tamanho do Arquivo Compactado}}$$

Algoritmos de Codificação

- Código de Huffman
- Run-Length
- Lempel-Ziv

E algumas variações desses.

Algoritmos de Compactação sem Perdas.

Código de Huffman

[1952]

- Variação do algoritmo de Shannon-Fano
 - Divide o conjunto de entrada em dois subconjuntos, atribuindo 0 a um deles e 1 ao outro
 - Prossegue com a codificação da mesma forma

Código de Huffman

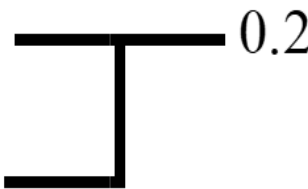
- Organizam-se os símbolos em ordem decrescente de suas probabilidades
- Uma fonte reduzida é formada a partir dos 2 símbolos de menor probabilidade (para codificação binária)
- Continua o passo anterior até que só restem 2 símbolos
- Atribui-se 0 a um dos símbolos e 1 ao outro, aleatoriamente
- Continua atribuindo 0's e 1's até chegar aos símbolos iniciais

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES
S1	0.3
S2	0.3
S3	0.2
S4	0.1
S5	0.1

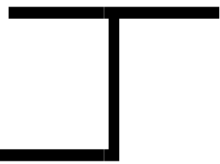
Exemplo: $S_1S_1S_1S_2S_2S_2S_3S_3S_4S_5$

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES	FONTE REDUZIDA
S1	0.3	
S2	0.3	
S3	0.2	
S4	0.1	
S5	0.1	

Junta os dois elementos de menores probabilidades...

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES	FONTE REDUZIDA
S1	0.3	0.3
S2	0.3	0.3
S3	0.2	0.2
S4	0.1	
S5	0.1	

Temos, assim, a primeira *fonte reduzida*.

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES	FONTE REDUZIDA	FONTE REDUZIDA	FONTE REDUZIDA
S1	0.3	0.3	0.4	0.6
S2	0.3	0.3	0.3	0.4
S3	0.2	0.2	0.3	
S4	0.1	0.2		
S5	0.1			

Continua com o mesmo processo até chegar a dois símbolos apenas....

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES	FONTE REDUZIDA	FONTE REDUZIDA	FONTE REDUZIDA
S1	0.3	0.3	0.4	0.6 1
S2	0.3	0.3	0.3	0.4 0
S3	0.2	0.2	0.3	
S4	0.1	0.2		
S5	0.1			

A partir dos últimos símbolos, atribui **0** para um deles e **1** para o outro, em qualquer ordem....

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES	FONTE REDUZIDA	FONTE REDUZIDA	FONTE REDUZIDA
S1	0.3	0.3	0.4 0	0.6 1
S2	0.3	0.3	0.3 1	0.4 0
S3	0.2	0.2	0.3 1	
S4	0.1	0.2		
S5	0.1			

Os códigos saem “voltando” de onde vieram...

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES	FONTE REDUZIDA	FONTE REDUZIDA	FONTE REDUZIDA
S1	0.3	0.3	0.4 0	0.6 1
S2	0.3	0.3	0.3 10	0.4 0
S3	0.2	0.2	0.3 11	
S4	0.1	0.2		
S5	0.1			

Onde se dividem, novamente, atribui, à direita, 0 para um e 1 para outro em qualquer ordem (não precisa manter a inicial).

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES	FONTE REDUZIDA	FONTE REDUZIDA	FONTE REDUZIDA
S1	0.3	0.3 10	0.4 0	0.6 1
S2	0.3	0.3 11	0.3 10	0.4 0
S3	0.2	0.2 0	0.3 11	
S4	0.1	0.2 0		
S5	0.1			

E segue da mesma forma....

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES	FONTE REDUZIDA	FONTE REDUZIDA	FONTE REDUZIDA
S1	0.3	0.3	10	0.4 0
S2	0.3	0.3	11	0.3 10
S3	0.2	0.2	00	0.3 11
S4	0.1	0.2	01	
S5	0.1			

E segue da mesma forma....

Código de Huffman

SÍMBOLOS DA FONTE	PROBABI- LIDADES	FONTE REDUZIDA	FONTE REDUZIDA	FONTE REDUZIDA
S1	0.3 10	0.3 10	0.4 0	0.6 1
S2	0.3 11	0.3 11	0.3 10	0.4 0
S3	0.2 00	0.2 00	0.3 11	
S4	0.1 010	0.2 01		
S5	0.1 011			

Até a codificação final....

Código de Huffman

- Observe que:
 - Cada símbolo tem sua própria codificação
 - O código gerado é unicamente decodificável e instantâneo
 - Os símbolos de maiores probabilidades recebem os códigos de menores comprimentos
 - Claro, garantindo que a codificação continue unicamente decodificável e com menor comprimento médio

Código de Huffman

- Codificação em 2 passos
 - Um para determinação das probabilidades
 - O outro para gerar a codificação
- Codificação *Bottom-Up*
- Variações:
 - Para Multi-Símbolos
 - Adaptativo
 - Truncado
 - Modificado

Codificação Aritmética

[1963]

- A codificação é baseada em intervalos de probabilidades de cada símbolo da fonte
- Cada símbolo da fonte será representado por um intervalo, onde, à medida que o intervalo se torna menor, o número de bits necessário para especificá-lo aumenta

Run-Length

Contagem de **carreiras** de pixels de mesma cor numa imagem



Sequência **#4#2#2**

- Variação:
 - Run-Length com *Sequência de Escape*

Run-Length

- No caso de imagens coloridas, é preciso armazenar a cor e a quantidade de vezes que ela se repete
 - Exemplo:
 - 120 120 120 30 45 45 45 60
 - Codificação:
 - $\langle 3, 120 \rangle \langle 1, 30 \rangle \langle 3, 45 \rangle \langle 1, 60 \rangle$
 - Isso pode provocar uma expansão no arquivo ao invés de compressão
 - Encontramos o run-length implementado no formato BMP_RLE (*Run Length Encoding*)

Algoritmo de Lempel-Ziv

[1977, 1978]

- Desenvolvido por Jakob Ziv e Abraham Lempel
- Codificação dinâmica
- Converte palavras de comprimento variável da entrada em códigos de comprimento fixo
- O algoritmo é universal

Esquema de Codificação

- Um dicionário é formado com as entradas e vai crescendo à medida que as palavras forem lidas e suas codificações geradas.

Esquema de Codificação

- Cada j nova palavra lida da entrada irá gerar uma palavra código de comprimento $\lceil \log_2(a.j) \rceil$, onde a é a base do alfabeto fonte
- Cada nova palavra lida deverá ser incluída como uma entrada no dicionário; caso a palavra já exista, um novo símbolo deve ser lido até formar uma palavra inédita

Esquema de Codificação

- A codificação é gerada através da representação em binário da concatenação de uma função ponteiro e de uma função de mapeamento
- A função ponteiro indica a palavra (caso exista) que serviu como prefixo para a atual entrada no dicionário

Esquema de Codificação

A codificação completa da j -ésima palavra é formada pelo par *código* mais a *função ponteiro*

Seja x uma palavra da entrada do codificador, a saída será dada por:

$$\langle C(x), P(x) \rangle$$

onde $C(x)$ é a codificação e $P(x)$ é a função ponteiro.

Lempel-Ziv Welch

[1984]

- Variação do Lempel-Ziv desenvolvida por Terry Welch
- Implementada no formato GIF para armazenamento de imagens
 - O formato GIF tem perdas em um certo sentido já que ele só armazena imagens com, no máximo, 256 cores

Lempel-Ziv Welch

Esquema de Codificação

- Inicializa o Dicionário
- [Prefixo] <- Vazio (Inicialização)
- C recebe o próximo símbolo no Código
- *[Prefixo]C* está no Dicionário?
 - SIM: [Prefixo] Recebe *[Prefixo]C* e volta a (3)
 - NÃO:
 - *[Prefixo]C* é acrescentado ao Dicionário
 - Saída recebe Código de [Prefixo]
 - [Prefixo] <- C e volta a (3)

Lempel-Ziv Welch Exemplo

Dicionário básico:

X = #0

Y = #1

Z = #2

W = #3

Essa codificação inicial do dicionário base poderia ser, por exemplo, o código ASCII.

Entrada do codificador:

XYXZXYXW

Lempel-Ziv Welch

Exemplo

Dicionário básico:

$X = \#0$, $Y = \#1$, $Z = \#2$, $W = \#3$

Entrada do codificador:

X YXZXYXW

Chegou X; faz parte do dicionário? Sim, logo:

$X \rightarrow \#0$

Lempel-Ziv Welch

Exemplo

Dicionário básico:

$X = \#0$, $Y = \#1$, $Z = \#2$, $W = \#3$

Entrada do codificador:

$XYXZXYXW$

Chegou Y; faz parte do dicionário? Sim, logo:

$X \rightarrow \#0$

$Y \rightarrow \#1$

Agora, checa se XY faz parte do dicionário.

Como não faz, ele entra no dicionário....

Lempel-Ziv Welch

Exemplo

Dicionário básico:

$X = \#0$, $Y = \#1$, $Z = \#2$, $W = \#3$

Entrada do codificador:

$XYXZXYXW$

Chegou X; faz parte do dicionário? Sim, logo:

$X \rightarrow \#0$

$Y \rightarrow \#1$

$X \rightarrow \#0$

Agora, checa se YX faz parte do dicionário....

Como não faz, ele entra no dicionário....

Lempel-Ziv Welch

Exemplo

Dicionário básico:

$X = \#0$, $Y = \#1$, $Z = \#2$, $W = \#3$

Entrada do codificador:

$XYXZ$ $XYXW$

Chegou Z; faz parte do dicionário? Sim, logo:

$X \rightarrow \#0$ $Y \rightarrow \#1$

$X \rightarrow \#0$ $Z \rightarrow \#2$

Agora, checa se XZ faz parte do dicionário....

Como não faz, ele entra no dicionário....

Lempel-Ziv Welch

Exemplo

Dicionário básico:

$X = \#0$, $Y = \#1$, $Z = \#2$, $W = \#3$

Entrada do codificador:

$XYXZX$ YXW

Chegou X; faz parte do dicionário? Sim, logo:

$X \rightarrow \#0$

$Z \rightarrow \#2$

$Y \rightarrow \#1$

$X \rightarrow \#0$

$X \rightarrow \#0$

Continua.....

Lempel-Ziv Welch

Exemplo

Dicionário básico:

$X = \#0$, $Y = \#1$, $Z = \#2$, $W = \#3$

Entrada do codificador:

$XYXZXYXW$

Chegou X; faz parte do dicionário? Sim, MAS XY também faz parte! Logo, a codificação fica a de XY

Lempel-Ziv Welch Exemplo

Codificação Final

#0#1#0#2#4#0#3

Dicionário

X = #0 Y = #1

Z = #2 W = #3

Dicionário Base

XY = #4 YX = #5

XZ = #6 ZX = #7

XYX = #8 XW = #9

Dicionário
Estendido

Lempel-Ziv Welch Exemplo

Decodificação

#0#1#0#2#4#0#3

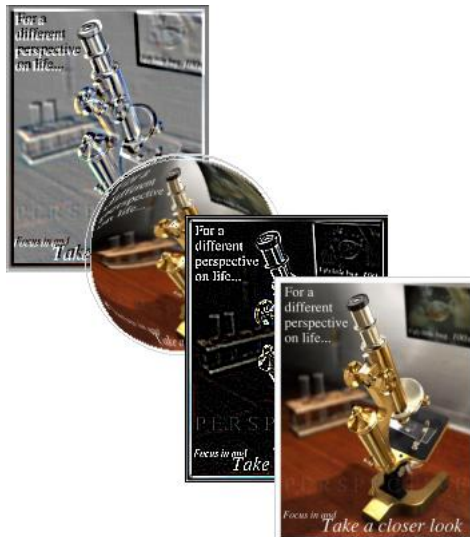
Dicionário Básico

X = #0 Y = #1

Z = #2 W = #3

Apenas a mensagem codificada e o dicionário base são necessários. A extensão do dicionário é criada da mesma forma que foi feito na codificação.

Formatos de Armazenamento de Arquivos de Imagem



Carlos Alexandre Barros de Mello
CIn/UFPE



Formatos de Armazenamento de Arquivos de Imagem

- BMP
- GIF
- JPEG
- TIFF
- WMF
- PIC
-

Formatos de Armazenamento de Arquivos de Imagem

- BMP
- GIF
- JPEG
- TIFF
- WMF
- PIC
-

Formatos de Armazenamento de Arquivos de Imagem

- Formatos criados para aplicações específicas:

- JPEG, GIF, PNG – Internet

- Altas taxas de compressão

- TIFF – Indústria Gráfica

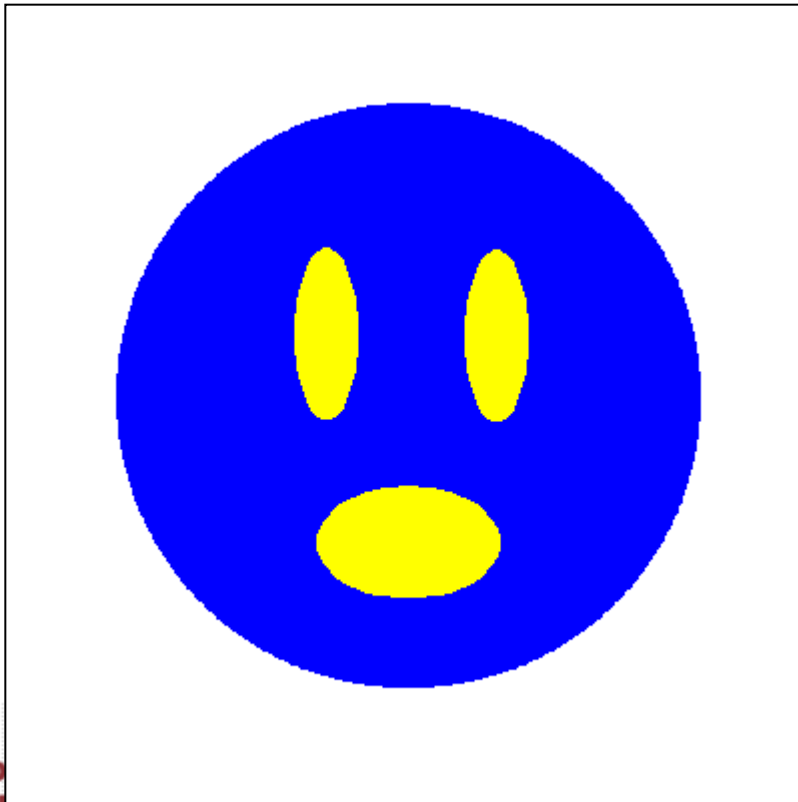
- Padrão de cores CMYK

Formatos de Armazenamento de Arquivos de Imagem

- Cuidado para não usar o formato errado na aplicação!
 - Exemplo: Internet
 - Qual o melhor formato para transmissão de imagens via Internet? JPEG?
 - Nem sempre!

Formatos de Armazenamento de Arquivos de Imagem

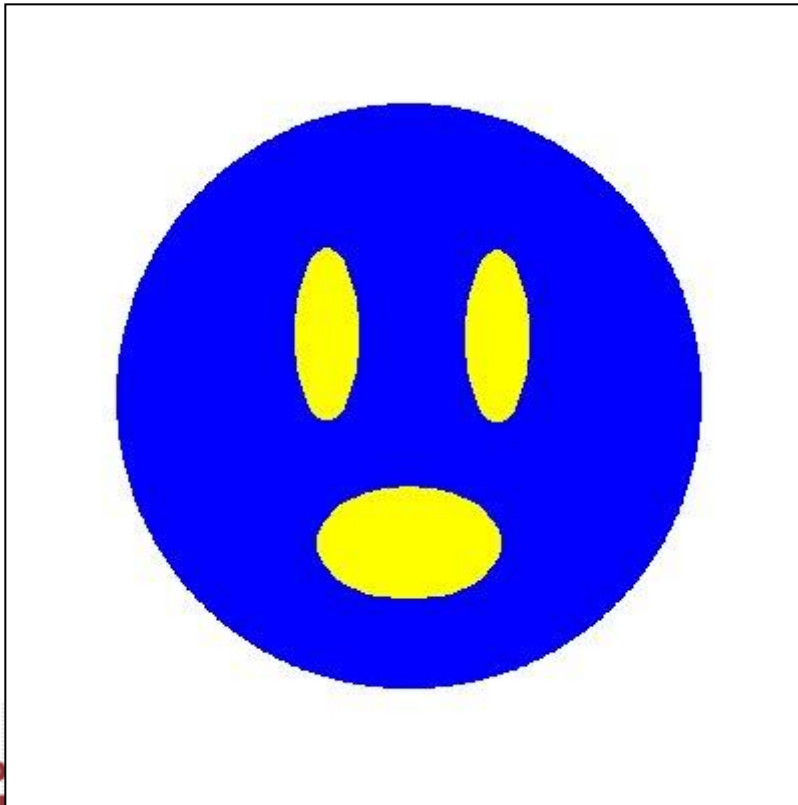
- Exemplo: Observe a seguinte imagem:



**BMP: 469KB
16M de cores
3 cores usadas**

Formatos de Armazenamento de Arquivos de Imagem

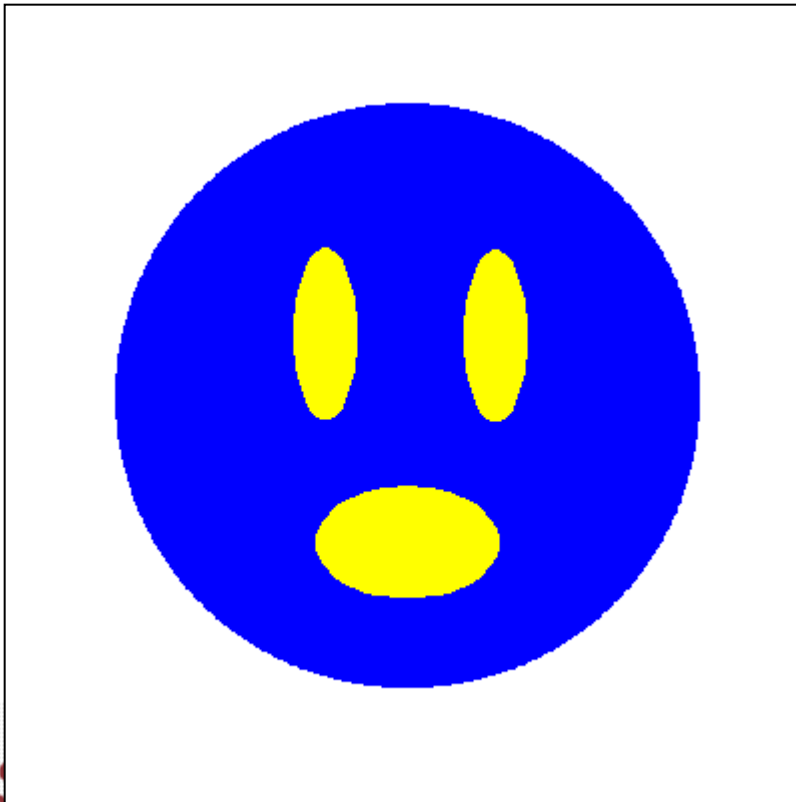
- Mesma imagem em JPG (15% de perda)



JPG: 14KB
16M de cores

Formatos de Armazenamento de Arquivos de Imagem

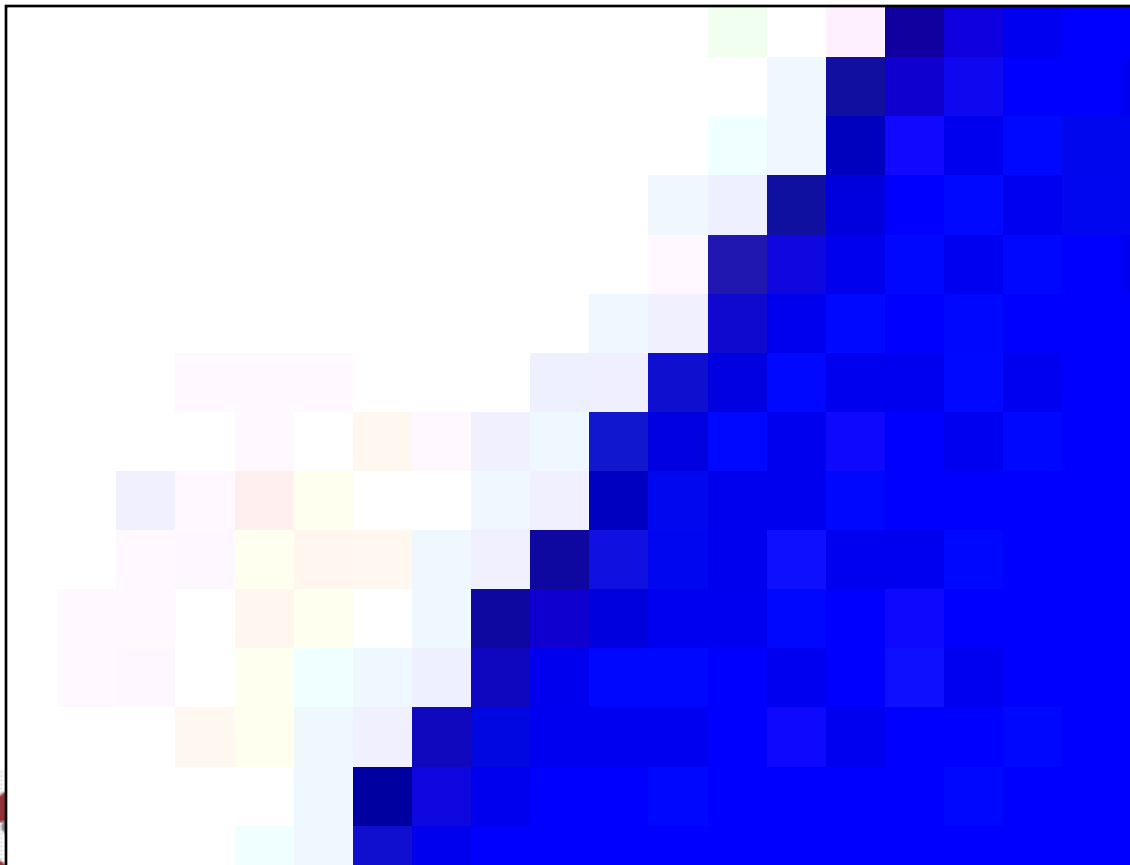
- Mesma imagem em GIF



**GIF: 3KB
256 cores**

Formatos de Armazenamento de Arquivos de Imagem

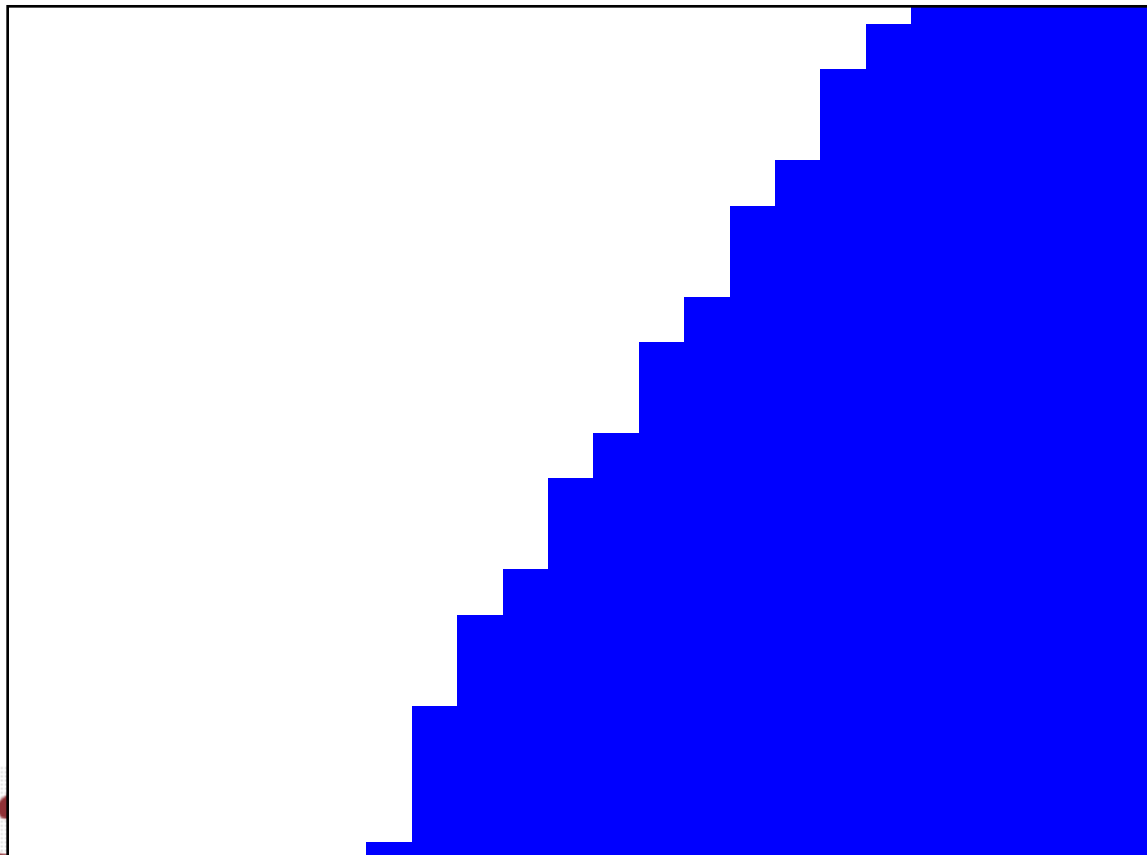
- A diferença não está só no tamanho do arquivo....



**Zoom na imagem
JPG....
Observe a quantidade
de cores**

Formatos de Armazenamento de Arquivos de Imagem

- A diferença não está só no tamanho do arquivo....



Zoom na imagem
GIF....
Apenas 3 cores...

Nem sempre JPG
é a melhor escolha!

Formatos de Armazenamento de Arquivos de Imagem

- Outro exemplo
 - Considere a imagem de texto abaixo (em tons de cinza)

From a graphical point of view, when analyzing a signal, it is always valuable to represent the different signals (s , A_j , D_j) and coefficients ($C(j,k)$).

Let us consider the Figure 6-7, on page 6-19.

On the left side, **s** is the signal, **a5**, **a4**, **a3**, **a2**, and **a1** are the approximations at level 5, 4, 3, 2, and 1. The best approximation is **a1**; the next one is **a2**, and so on. Noise oscillations are exhibited in **a1**, whereas **a5** is smoother.

On the right side, **cfs** represents the coefficients (for more information, see “Wavelet Transforms: Continuous and Discrete” on page 6-13), **s** is the signal, **d5**, **d4**, **d3**, **d2**, and **d1** are the details at level 5, 4, 3, 2, and 1.

The different signals that are presented exist in the same time grid. We can consider that the t index of detail $D_4(t)$ identifies the same temporal instant as that of the approximation $A_5(t)$ and that of the signal $s(t)$. This identity is of considerable practical interest in understanding the composition of the signal, even if the wavelet sometimes introduces dephasing.

Formatos de Armazenamento de Arquivos de Imagem

- Outro exemplo
 - Veja o resultado ao salvarmos ela em JPEG (40% de perda)

From a graphical point of view, when analyzing a signal, it is always valuable to represent the different signals (s , A_j , D_j) and coefficients ($C(j,k)$).

Let us consider the Figure 6-7, on page 6-19.

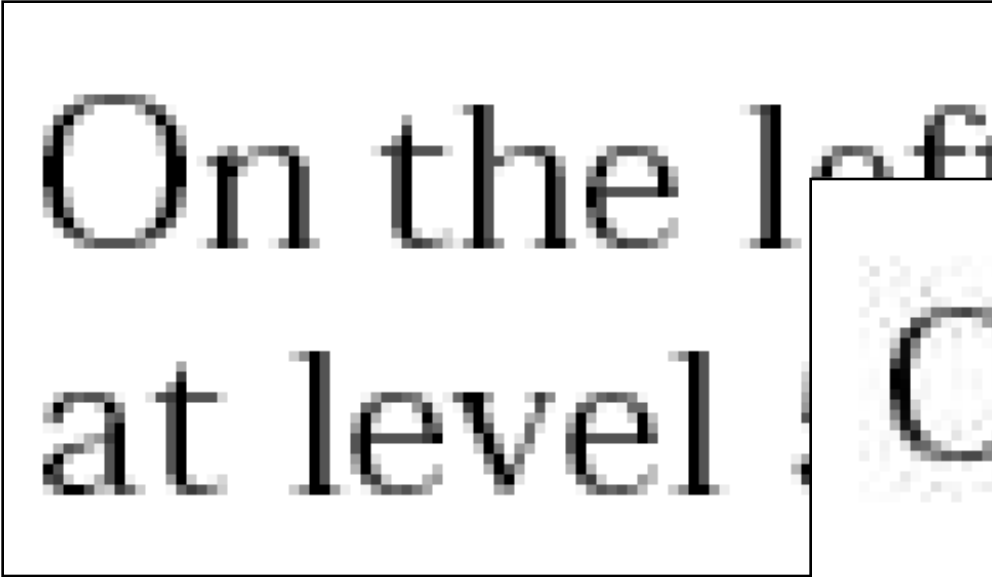
On the left side, s is the signal, a_5 , a_4 , a_3 , a_2 , and a_1 are the approximations at level 5, 4, 3, 2, and 1. The best approximation is a_1 ; the next one is a_2 , and so on. Noise oscillations are exhibited in a_1 , whereas a_5 is smoother.

On the right side, cfs represents the coefficients (for more information, see “Wavelet Transforms: Continuous and Discrete” on page 6-13), s is the signal, d_5 , d_4 , d_3 , d_2 , and d_1 are the details at level 5, 4, 3, 2, and 1.

The different signals that are presented exist in the same time grid. We can consider that the t index of detail $D_4(t)$ identifies the same temporal instant as that of the approximation $A_5(t)$ and that of the signal $s(t)$. This identity is of considerable practical interest in understanding the composition of the signal, even if the wavelet sometimes introduces dephasing.

Formatos de Armazenamento de Arquivos de Imagem

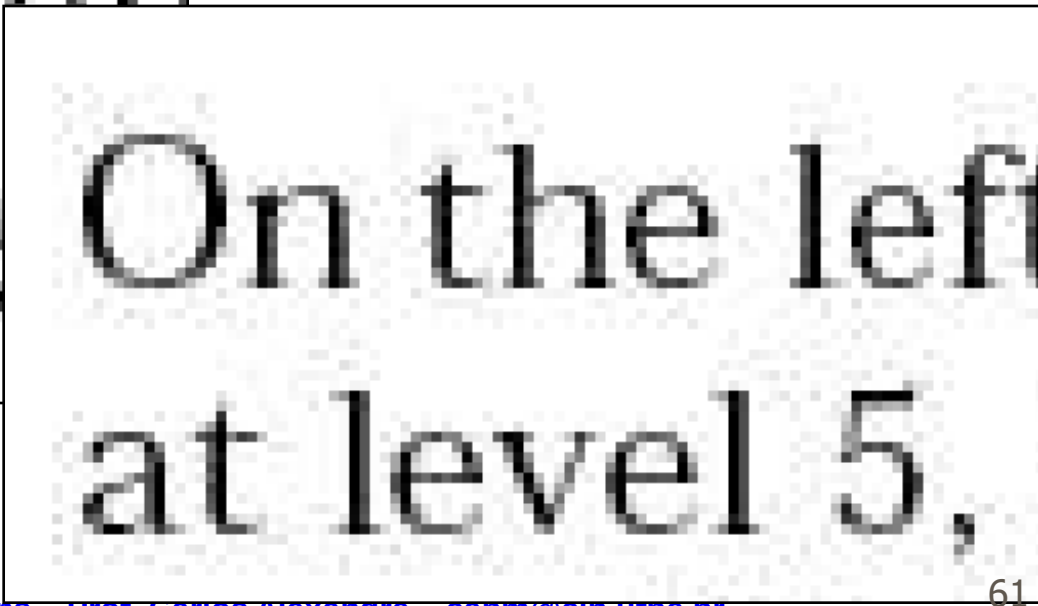
- Outro exemplo
 - Veja em detalhes.....



On the left
at level

Imagem Original

Imagem JPEG

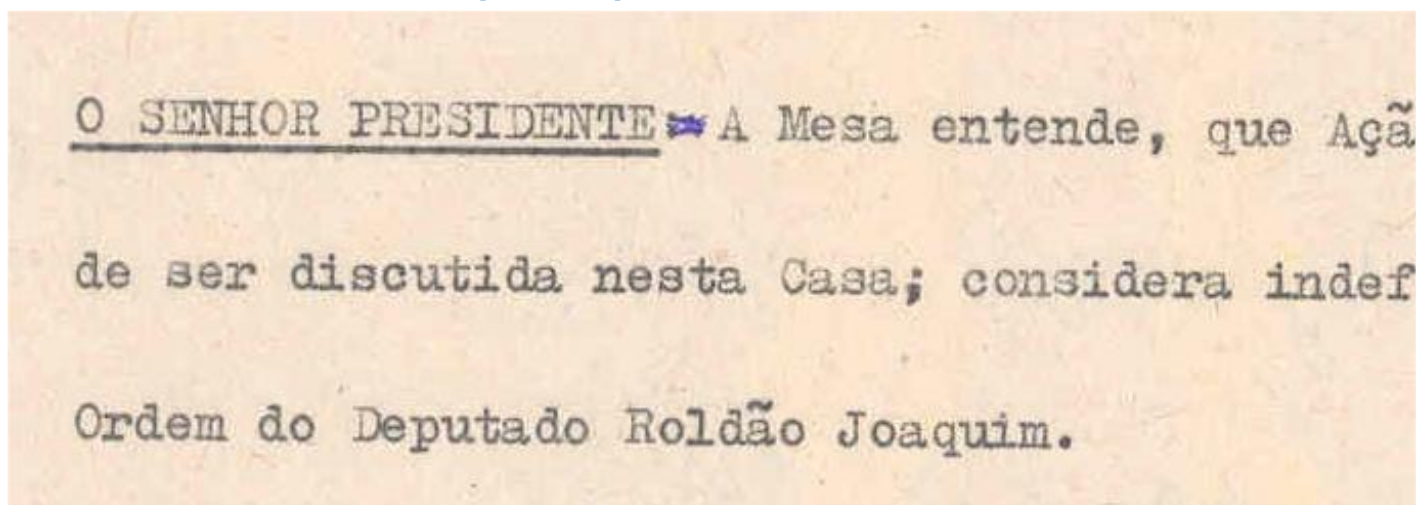


On the left
at level 5,

Formatos de Armazenamento de Arquivos de Imagem

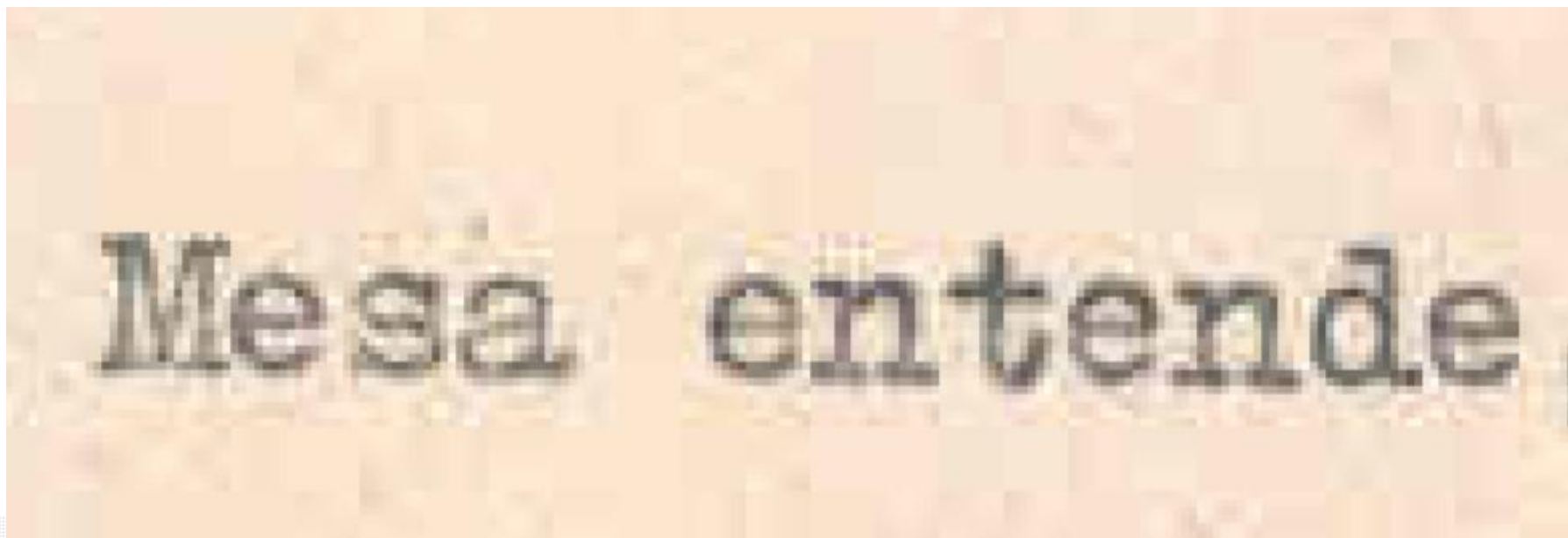
- Outro exemplo

- Mesmo exemplo em uma situação real:
- Esse documento foi digitalizado com a perda padrão do formato JPG (15%) – definida no scanner



Formatos de Armazenamento de Arquivos de Imagem

- Outro exemplo
 - Apesar de ser um texto tipografado, nenhum OCR, na época, conseguiu reconhecer o texto....



Formatos de Armazenamento de Arquivos de Imagem

- BMP (*.bmp):
 - Formato Padrão do Microsoft Windows
 - Qualquer tipo de imagem (cores)
 - 2 Modos:
 - Sem Compactação
 - Com Compactação (Run-Length)
 - Cabeçalho de 52 bytes (+ paleta de cores)
 - Sem compressão

Formatos de Armazenamento de Arquivos de Imagem

- BMP (*.bmp): Cabeçalho
 - Assinatura (BM)
 - Dimensões da Imagem (X e Y)
 - Início da Imagem
 - Número de Cores (2, 16, 256, 16M)
 - etc

Formatos de Armazenamento de Arquivos de Imagem

- BMP (*.bmp): Observações
 - Imagem armazenada ao contrário
 - Cores:
 - 2 cores: Cada pixel é representado em 1 bit
 - 16 cores: Pixels representados em 4 bits
 - 256 cores: Pixels representados em 1 byte
 - 16M cores: Pixels representados em 3 bytes
 - Variação com compressão: BMP_RLE

Formatos de Armazenamento de Arquivos de Imagem

- GIF (*.gif): *Graphics Interchange Format*
 - Desenvolvido pela CompuServe e propriedade da Unisys
 - Formato com Compressão *Sem Perdas* (Lempel-Ziv-Welch)
 - Trabalha apenas com imagens com 256 cores ou menos
 - Permite uso de cores *transparentes*

Formatos de Armazenamento de Arquivos de Imagem

- GIF (*.gif): Graphics Interchange Format
 - Permite armazenamento de mais de uma imagem em um mesmo arquivo
 - Uso de *Mapa de Cores Global* e um *Mapa de Cores Local*
 - *Caracteres separadores de imagens*

Formatos de Armazenamento de Arquivos de Imagem

- JPEG (*.jpg): Joint Photographic Experts Group
 - Compressão *Com Perdas*
 - A Taxa de Perda define a qualidade da imagem
 - Só trabalha com imagens com 16M de cores (ou em 256 tons de cinza)
 - Implementável a nível de SW ou HW

Formatos de Armazenamento de Arquivos de Imagem

- JPEG (*.jpg): Algumas informações
 - Atua bem em imagens de fotografias (não desenhos) porque foi criado para lidar com características do sistema visual humano
 - Usa o modelo de cor YCbCr, por ser mais próximo ao SVH
 - Tem controle de degradação
 - Quanto maior a degradação, pior a qualidade da imagem, mas menor o tamanho do arquivo

Formatos de Armazenamento de Arquivos de Imagem

- JPEG (*.jpg): Codificação – Principais passos:

- Transformação de cor: RGB $\rightarrow$ YCbCr

- RGB $\rightarrow$ YCbCr

- $Y = 0,299R + 0,587G + 0,114B$

- $Cb = -0,1687R - 0,3313G + 0,5B + 128$

- $Cr = 0,5R - 0,4187G - 0,0813B + 128$

- $(120; 60; 30)_{\text{RGB}} \rightarrow (74,52; 102,88; 160,44)_{\text{YCbCr}}$

Formatos de Armazenamento de Arquivos de Imagem

- JPEG (*.jpg): Codificação – Principais passos:
 - Transformação de cor: RGB $\rightarrow$ YCbCr
 - Down Sampling: 2H 1V
 - 2:1 na horizontal e 1:1 na vertical
 - Agrupamentos de 8x2 são criados
 - DCT é aplicada a cada grupo (saída: 8x8)
 - Quantização
 - Codificação (RLE + Huffman)

Formatos de Armazenamento de Arquivos de Imagem

- JPEG (*.jpg): Codificação – Principais passos:
 - Detalhes sobre a estrutura do arquivo:
 - <https://docs.fileformat.com/image/jpeg/>

Formatos de Armazenamento de Arquivos de Imagem

- JPEG (*.jpg): 80% de perda



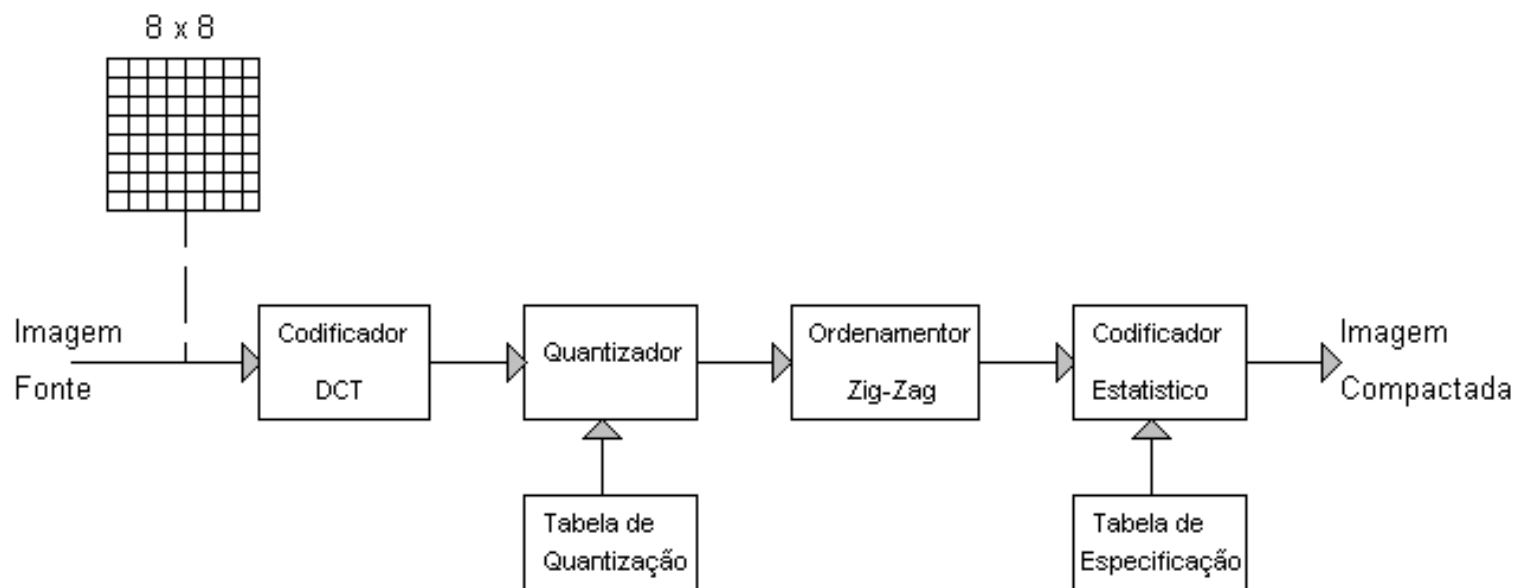
Formatos de Armazenamento de Arquivos de Imagem

- JPEG

- Suporta 4 Modos de Operação
 - Sequencial
 - Progressiva
 - Sem Perda
 - Hierárquica
- Uso da DCT - *Discrete Cosine Transform*

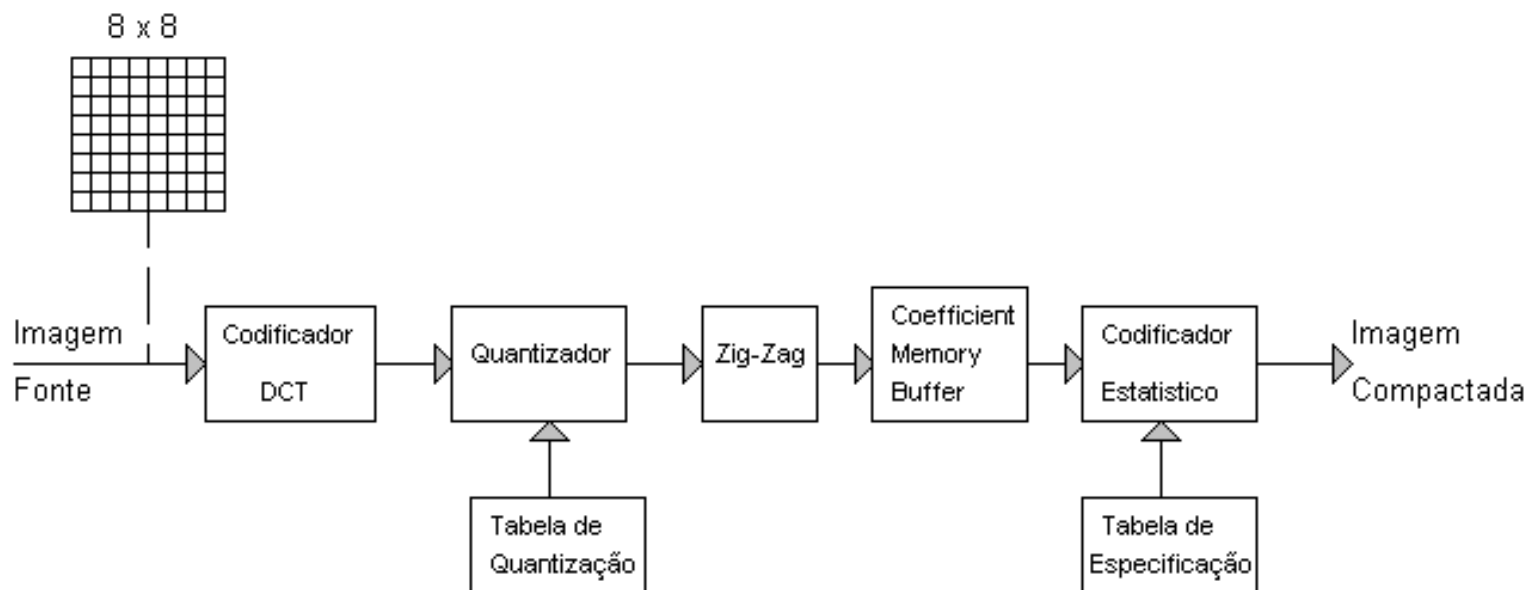
Formatos de Armazenamento de Arquivos de Imagem

- JPEG
 - Codificação Sequencial



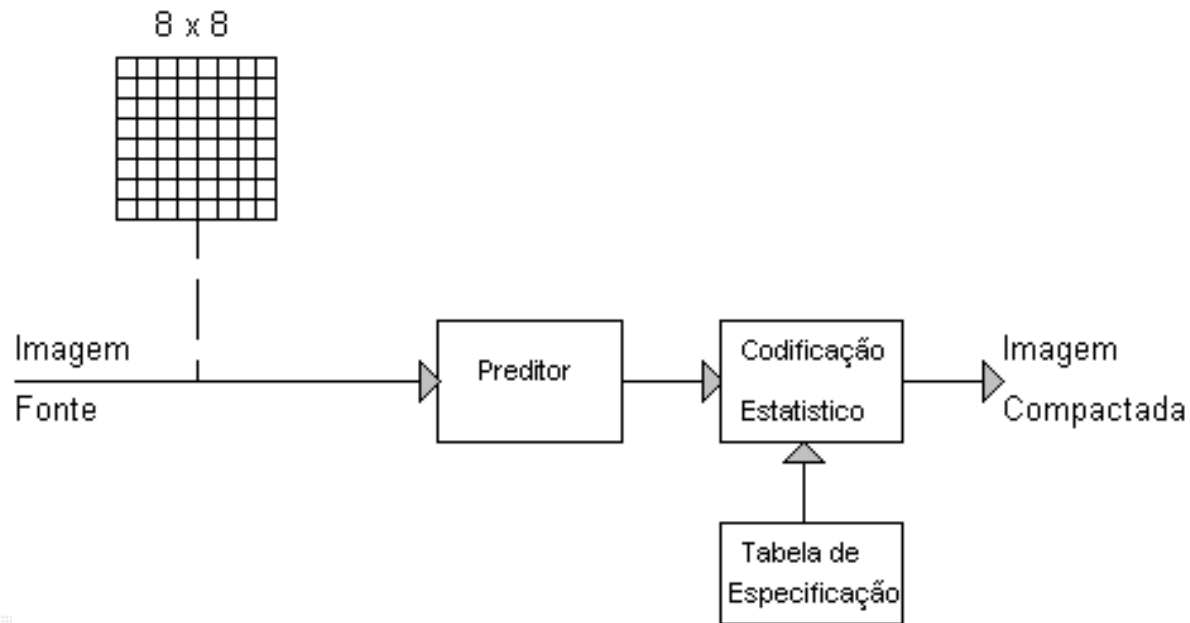
Formatos de Armazenamento de Arquivos de Imagem

- JPEG
 - Codificação Progressiva



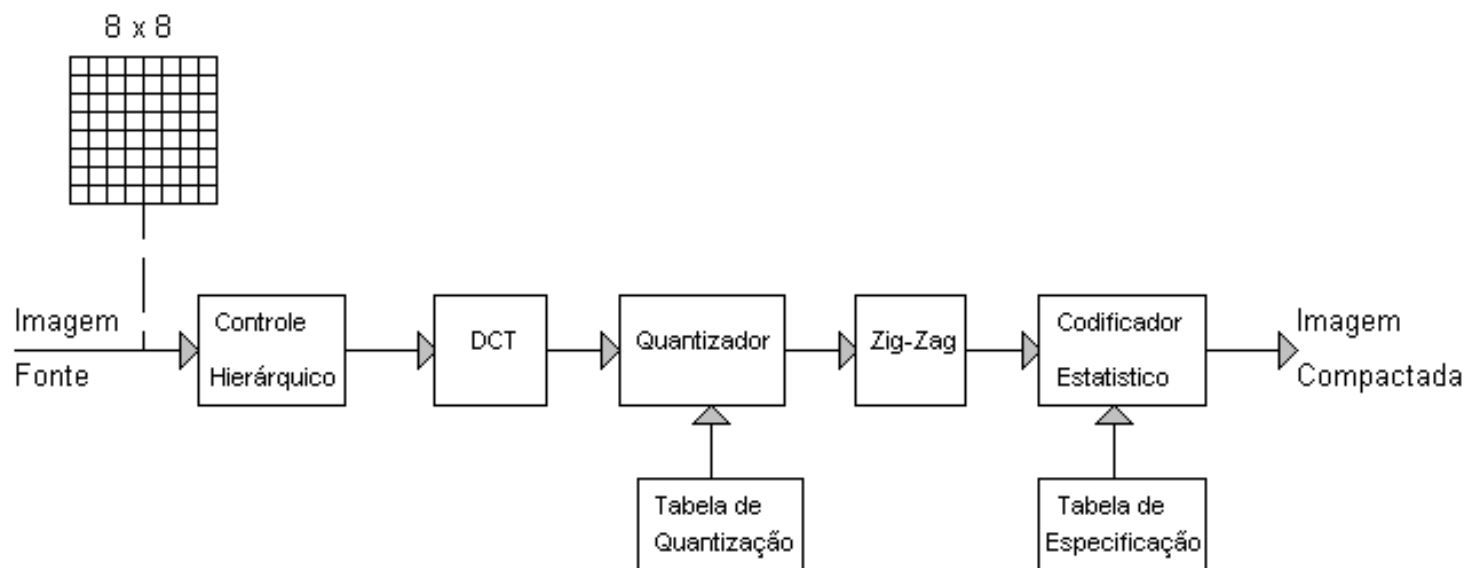
Formatos de Armazenamento de Arquivos de Imagem

- JPEG
 - Codificação Sem Perda



Formatos de Armazenamento de Arquivos de Imagem

- JPEG
 - Codificação Hierárquica



Formatos de Armazenamento de Arquivos de Imagem

- A DCT faz uma transformação na imagem, mudando o domínio de representação da mesma
- Este processo não introduz perdas de qualidade na imagem, sua utilização se dá porque ela permite uma representação mais compacta da imagem, facilitando a compressão
- O uso da DCT faz com que as maiores frequências se concentrem no canto superior esquerdo da matriz

Formatos de Armazenamento de Arquivos de Imagem

- Após a aplicação da DCT, uma quantização é feita tomando uma matriz de quantização fixa
- Cada valor da matriz gerada pela DCT é dividido pelo valor correspondente na máscara de quantização e arredondado para um inteiro

Formatos de Armazenamento de Arquivos de Imagem

- JPEG

- DCT + Quantização + Run-Length

64,24	47,74	42,63	34,95	30,81	25,87	21,82	19,78
45,12	42,20	37,01	31,34	27,70	23,13	19,93	17,53
38,55	34,55	30,83	28,21	23,04	19,23	16,91	15,89
28,61	28,22	26,30	22,78	18,86	16,10	15,08	14,30
23,59	22,69	21,11	18,71	15,72	12,93	13,01	11,47
18,15	16,86	15,99	14,62	12,87	11,74	11,13	10,98
15,83	14,04	12,44	11,29	10,37	10,89	11,00	10,46
13,36	10,67	10,09	9,41	9,88	10,67	10,99	11,46

Bloco da imagem (YCbCr)

168	45	7	3	2	2	1	1
67	32	3	3	2	1	1	1
12	5	5	5	2	1	2	1
5	5	2	2	1	1	1	1
3	2	2	1	1	1	1	1
2	2	2	1	1	1	1	0
1	1	1	1	2	1	1	0
1	2	1	1	1	1	0	0

Resultado após DCT

Formatos de Armazenamento de Arquivos de Imagem

- JPEG

- DCT + Quantização + Run-Length

1	1	1	1	1	4	8	16
1	1	1	1	4	4	8	16
1	1	1	2	4	4	8	16
2	8	8	8	8	16	16	16
4	8	8	8	8	16	16	32
4	8	8	8	16	16	16	32
4	8	8	8	16	16	32	32
8	8	8	16	16	32	32	64

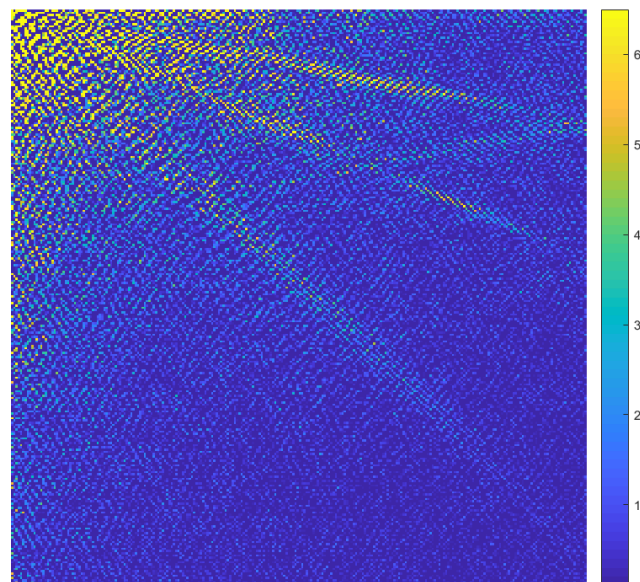
Matriz de quantização

168	45	7	3	2	0	0	0
67	32	3	3	0	0	0	0
12	5	5	2	0	0	0	0
2	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Bloco quantizado

Formatos de Armazenamento de Arquivos de Imagem

- DCT + Quantização + Run-Length
 - DCT (re-lembrando)
 - Exemplo: imagem inteira



Formatos de Armazenamento de Arquivos de Imagem

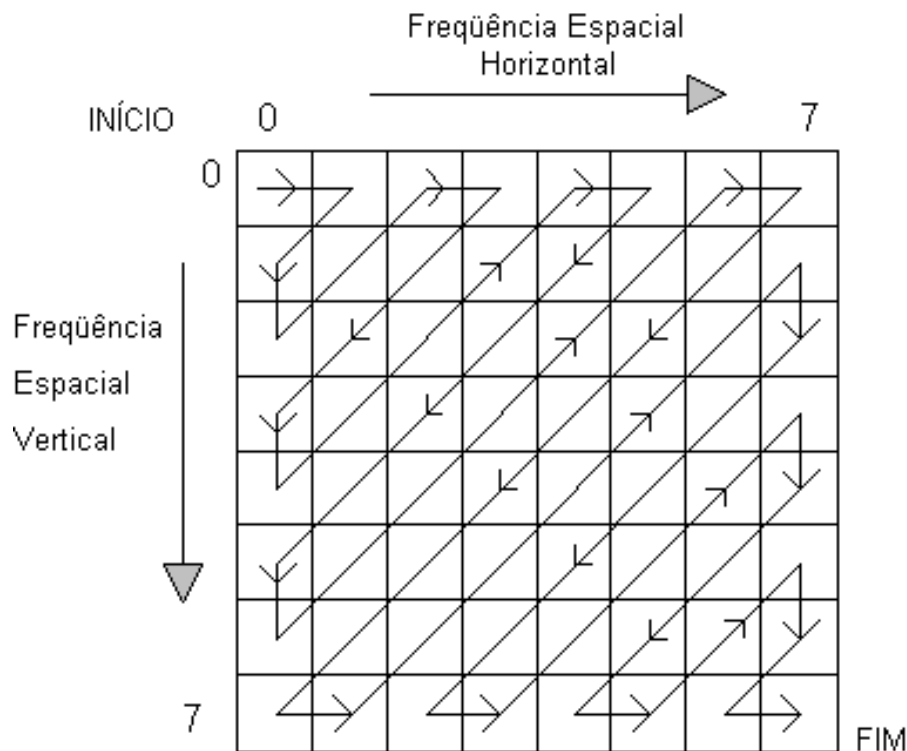
- DCT + Quantização + Run-Length
 - iDCT
 - Perdas



Formatos de Armazenamento de Arquivos de Imagem

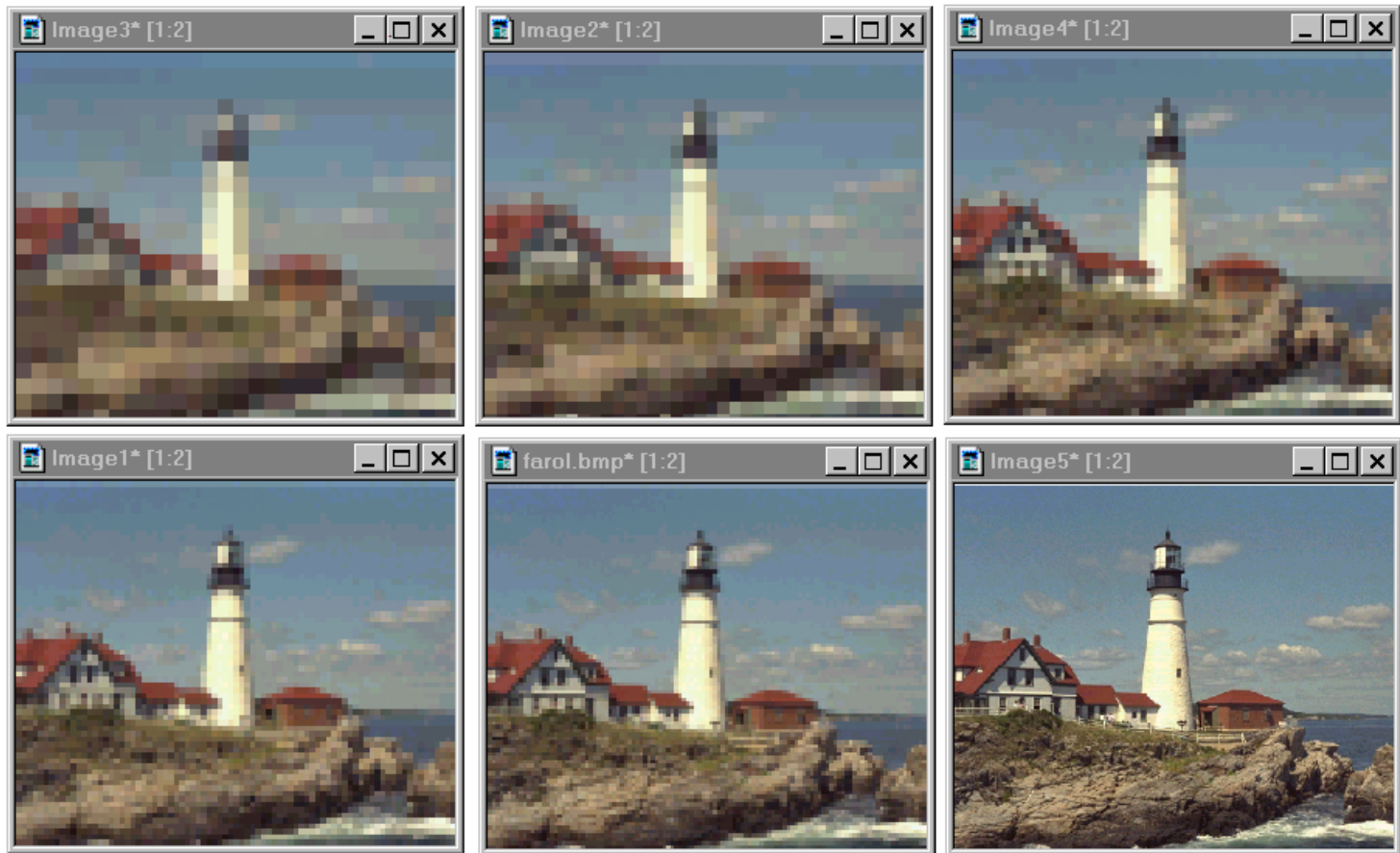
● JPEG

- Ordenação em Zig-Zag dos componentes da DCT a aumentar a eficiência da codificação estatística



Formatos de Armazenamento de Arquivos de Imagem

- JPEG (*.jpg): Modo Progressivo



Formatos de Armazenamento de Arquivos de Imagem

- Exemplo:



Formatos de Armazenamento de Arquivos de Imagem

- Exemplo:
 - BMP: 800.934 bytes
 - GIF (256 cores): 120.039 bytes
 - JPG (sequencial)
 - (sem perda): 491.238 bytes
 - (15% de perda): 73.505 bytes
 - (50% de perda): 32.067 bytes
 - JPG (progressivo - 15% de perda): 69.540 bytes
 - TIFF, PNG,

Referências

- Introduction to Data Compression, Khalid Sayood

